

DOCUMENTATION TECHNIQUE

Documentation Technique & Architecture du Pipeline

Table des Matières

- CLI To-Do List Manager - Technical Specification & Architecture Document 3**
 - 1. Executive Summary & Architecture Overview 3
 - 1.1 Executive Brief 3
 - 1.2 Maturity Assessment 3
 - 1.3 Technical Stack 3
 - 1.4 Architectural Constraints 3
 - 1.5 Critical Dependencies 3
 - 2. Architecture Workflows 4
 - 2.1 Project Implementation Roadmap & Traceability 4
 - 2.2 CLI Command Execution Workflow 4
 - 3. Detailed Technical Specifications & Business Rules 5
 - 3.1 Requirements Traceability 5
 - 3.2 Security Rules 7
 - 3.3 Data Models 7
 - 4. Project Governance & Structural Gaps 7
 - 4.1 Structural Gaps 7
 - 4.2 Remediation & Workflow 7
 - 5. Technical & Domain Glossary (Terminology Reference) 8

CLI To-Do List Manager - Technical Specification & Architecture Document

1. Executive Summary & Architecture Overview

1.1 Executive Brief

The project is a Python-based CLI To-Do List Manager utilizing a local JSON file system for persistence. It implements a service-oriented architecture separating CLI dispatch, business logic (service layer), and data persistence (storage layer). The system enables task lifecycle management including creation, sequential ID assignment, completion tracking, and bulk clearing of finished tasks.

1.2 Maturity Assessment

The specification is logically sound and highly structured, achieving a high readiness state. However, the lack of formal acceptance criteria and the absence of defined security/performance constraints for file I/O require minor attention. Overall, the project is READY for execution as the implementation roadmap is comprehensive.

1.3 Technical Stack

- Python
- pyproject.toml

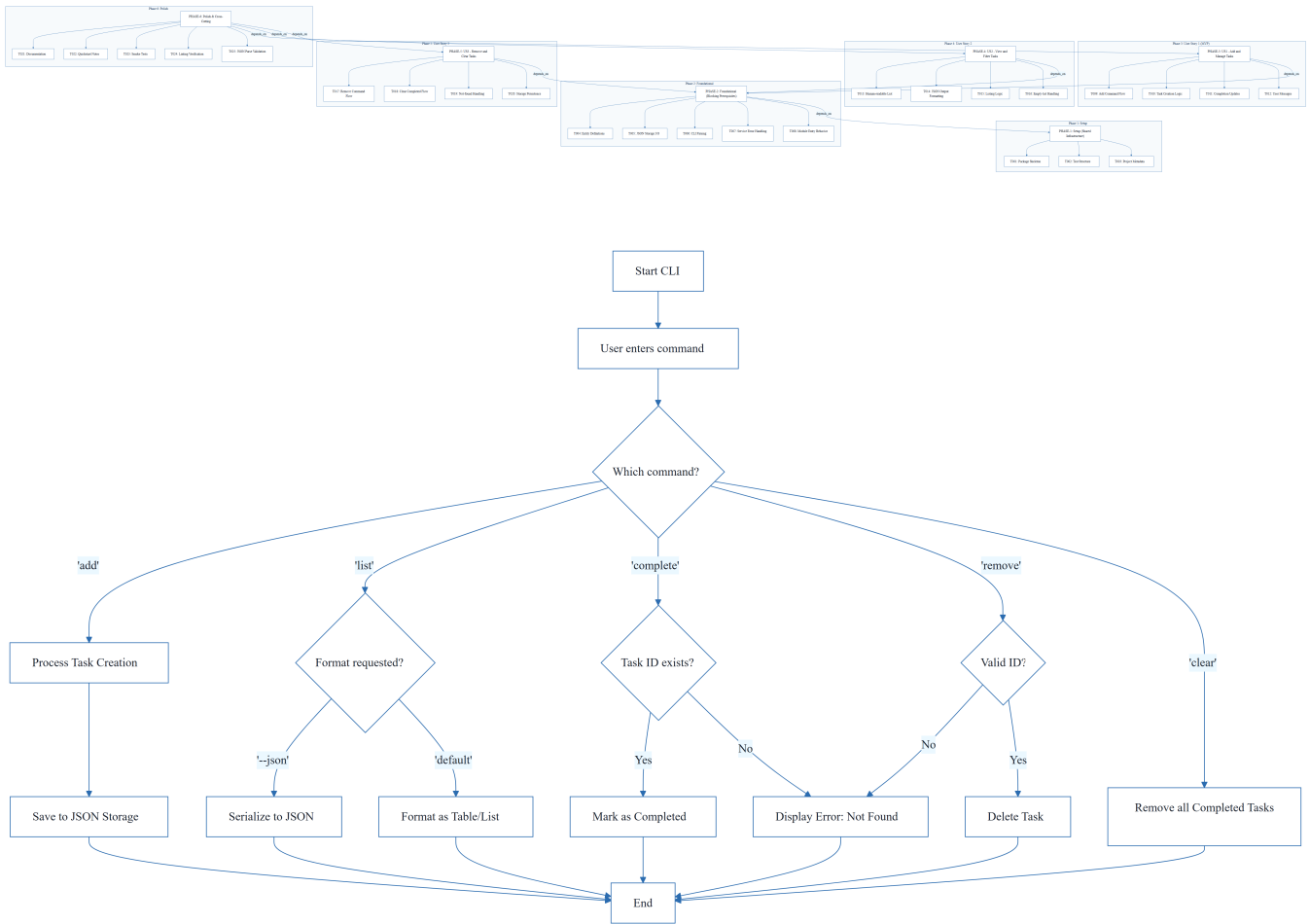
1.4 Architectural Constraints

- Sequential ID assignment for all created tasks.
- Storage persistence strictly via JSON file at `~/ . todos . json`.
- Strict execution order: Setup $\rightarrow$ Foundational $\rightarrow$ User Stories $\rightarrow$ Polish.
- JSON output must remain parseable for the `--json` flag.
- Foundational phase is a blocking prerequisite for all user story implementation.

1.5 Critical Dependencies

- JSON storage path resolution and file I/O helpers in `src/todo_manager/storage.py`.
- Task entity serialization helpers in `src/todo_manager/models.py`.
- `~/ . todos . json` filesystem access for data persistence.
- Dependency chain: Phase 1 (Setup) $\rightarrow$ Phase 2 (Foundational) $\rightarrow$ Phases 3, 4, 5 (User Stories).
- Manual smoke test gate for add, list, complete, remove, and clear commands.

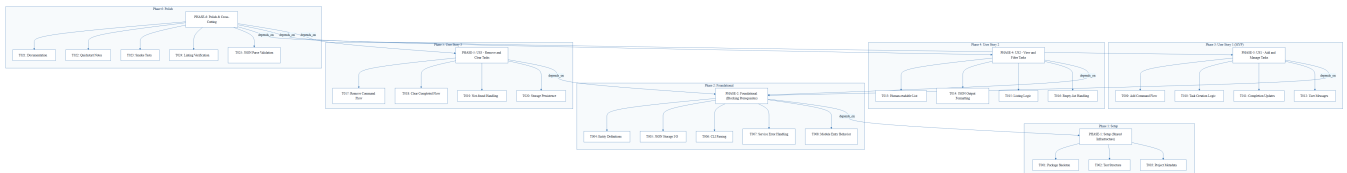
2. Architecture Workflows



& Visual Diagrams

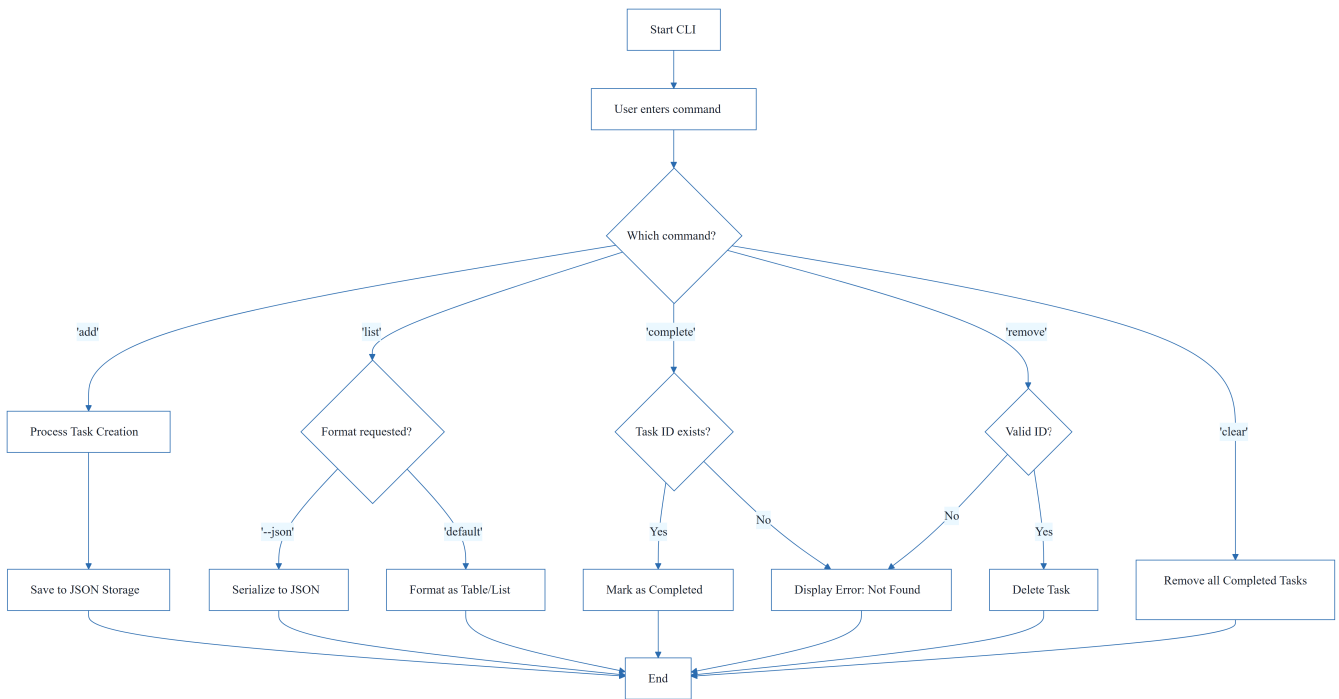
2.1 Project Implementation Roadmap & Traceability

Visualizes the phase dependencies and the mapping of specific tasks to their respective project phases.



2.2 CLI Command Execution Workflow

Models the business logic flow for the CLI manager, including decision points for different command types.



3. Detailed Technical Specifications & Business Rules

3.1 Requirements Traceability

ID	Phase/Story	Description	Source File/Context
T001	PHASE-1	Create the Python package skeleton	src/todo_manager/
T002	PHASE-1	Create the test directory structure	tests/unit/, tests/integration/
T003	PHASE-1	Add project metadata and tooling entry points	pyproject.toml
T004	PHASE-2	Implement task entity definitions and serialization helpers	src/todo_manager/models.py
T005	PHASE-2	Implement JSON storage path resolution and file I/O helpers	src/todo_manager/storage.py
T006	PHASE-2	Implement shared command-line parsing and top-level dispatch	src/todo_manager/cli.py
T007	PHASE-2	Implement shared service-layer error handling and task collection utilities	src/todo_manager/service.py

ID	Phase/Story	Description	Source File/Context
T008	PHASE-2	Define module entry behavior for python -m todo_manager	src/todo_manager/__main__.py
T009	US1	Implement the add command flow	cli.py, service.py
T010	US1	Implement task creation, sequential ID assignment, and created_at population	src/todo_manager/models.py
T011	US1	Implement completion updates for existing tasks	src/todo_manager/service.py
T012	US1	Add user-facing success and error messages for add and complete operations	src/todo_manager/cli.py
T013	US2	Implement human-readable task listing output	src/todo_manager/cli.py
T014	US2	Implement --json output formatting	src/todo_manager/cli.py
T015	US2	Add listing logic that preserves task order and status fields	src/todo_manager/service.py
T016	US2	Handle the empty-list case with a clear message	src/todo_manager/cli.py
T017	US3	Implement the remove command flow	cli.py, service.py
T018	US3	Implement the clear command flow for removing completed tasks	src/todo_manager/service.py
T019	US3	Add not-found handling for invalid task IDs	src/todo_manager/cli.py
T020	US3	Ensure storage writes persist removals and clears safely	src/todo_manager/storage.py
T021	PHASE-6	Document the CLI usage and storage behavior	README.md
T022	PHASE-6	Add quickstart verification notes and examples	specs/001-cli-todo-manager/quickstart.md
T023	PHASE-6	Manual smoke test of add, list, complete, remove, and clear	~/todos.json
T024	PHASE-6	Verify linting and formatting expectations	src/todo_manager/

ID	Phase/Story	Description	Source File/Context
T025	PHASE-6	Confirm the generated JSON output remains parseable	todo list --json

3.2 Security Rules

- **File Access:** The application must have read/write permissions for the home directory to manage `~/todos.json`.
- **Data Integrity:** Storage writes must be atomic or safely handled to prevent JSON corruption during `remove` or `clear` operations (T020).

3.3 Data Models

- **Task Entity:** Defined in `src/todo_manager/models.py`.
- **Attributes:** Includes sequential unique ID, task description, completion status, and `created_at` timestamp.
- **Persistence:** Serialized to JSON format.

4. Project Governance & Structural Gaps

4.1 Structural Gaps

Gap	Priority	Remediation Advice
Acceptance Criteria	MEDIUM	Add a formal list of acceptance criteria for each user story instead of relying only on 'Independent Test' descriptions.
Security & Performance Constraints	LOW	Define constraints for file access permissions and JSON file size limits.
Open Questions & Uncertainties	LOW	No open questions were identified in the source; ensure none exist before implementation.

4.2 Remediation & Workflow

The project follows an incremental delivery model:

1. **Setup $\rightarrow$ Foundational:** Establish the core architecture.
2. **US1 (MVP):** Validate basic add/complete behavior.
3. **US2 $\rightarrow$ US3:** Iteratively add viewing and deletion capabilities.
4. **Polish:** Finalize documentation and smoke tests.

5. Technical & Domain Glossary (Terminology Reference)

Term	Category	Context Anchor	Project Definition
Checkpoint	TECHNICAL_STACK	PHASE-2	A validation gate ensuring specific infrastructure is operational before dependent user stories commence.
Foundational	TECHNICAL_STACK	PHASE-2	The core architectural layer providing blocking shared utilities and base logic for all subsequent features.
Goal	BUSINESS_DOMAIN	PHASE-3	The primary objective defining the success criteria for a specific user-facing feature set.
ID	TECHNICAL_STACK	T010	A sequential unique alphanumeric token used to target specific entries within the persistent store.
JSON	TECHNICAL_STACK	T005	The standardized lightweight data-interchange format used for local persistent storage and programmatic output.
MVP	BUSINESS_DOMAIN	PHASE-3	The minimal set of functional capabilities required to demonstrate a viable working product.
Organization	BUSINESS_DOMAIN	Tasks: CLI To-Do List Manager	The structural grouping of development efforts based on independent user requirements.
Prerequisites	TECHNICAL_STACK	Tasks: CLI To-Do List Manager	External specification documents required to be finalized before execution of the implementation roadmap.
README	TECHNICAL_STACK	T021	The primary documentation file detailing usage instructions and architectural behavior.
Setup	TECHNICAL_STACK	PHASE-1	The initial phase encompassing package skeleton creation and environment configuration.

Term	Category	Context Anchor	Project Definition
T016	TECHNICAL_STACK	T016	The specific implementation task for managing empty-state feedback in the interface.
Tests	TECHNICAL_STACK	T002	The verification suite composed of unit and integration directory structures to ensure logic correctness.
population in	TECHNICAL_STACK	T010	The process of assigning an initial timestamp value to the creation metadata of an entity.
todo clear	BUSINESS_DOMAIN	T018	The operational logic used to purge all entries marked as finished from the system.
todo list	BUSINESS_DOMAIN	T013	The retrieval and display operation for the entire collection of stored entities.
■■ CRITICAL	TECHNICAL_STACK	PHASE-2	A high-severity constraint marking a phase that must be fully completed before any other work can proceed.